โครงงาน: GutHib

(แพลตฟอร์มบริหารจัดการงานสไตล์ Repository)

เสนอ

อาจารย์ ผศ.วีรยุทธ เจริญเรืองกิจ

โดย

1. นายอภินัทธ์ ยศวราเลิศ รหัส: 67102010178
2. นายชัยสิทธิ์ ปลื้มสุทธิ์ รหัส: 67102010510
3. นายพลภัทร เพ็ชรสินธพ รหัส: 67102010523

โครงงานนี้เป็นส่วนหนึ่งของการศึกษารายวิชา CP456 SOFTWARE ENGINEERING

มหาวิทยาลัยศรีนครินทรวิโรฒ

ภาคการศึกษาที่ 2 ปีการศึกษา 2567

สารบัญ

หัวข้อ	หน้า
ที่มาของปัญหาและความสำคัญ	1
วัตถุประสงค์ของโครงการ	1
ประโยชน์ที่คาดว่าจะได้รับ	2
ขอบเขตของโครงงาน	2
Requirement	2
Functional Requirement	2
Non-Functional Requirement	3
Requirement ที่เปลี่ยนไป	3
Design Document	4
Use Case Diagram	4
สิ่งที่เปลี่ยนแปลง (จาก Phase 1)	5
กระบวนการทำงานใหม่ (Process, Methods, and Tools)	6
Sprint Duration และ Scrum Events	6
เครื่องมือ Track Status งาน	6
Branch Naming Convention	7
Commit Message Format	7
ช่องทางการสื่อสารในทีม	8

ที่มาของปัญหาและความสำคัญ

GutHib ได้รับแรงบันดาลใจมาจากความซับซ้อนในการใช้งาน GitHub สำหรับบุคคลทั่วไปหรือสายงานอื่นที่ไม่ใช่นักพัฒนาซอฟต์แวร์ แม้ว่า GitHub จะมีประสิทธิภาพสูงในการจัดการงานเป็นทีม แต่ผู้ใช้ทั่วไปมักประสบปัญหาด้านความเข้าใจในการใช้งาน เนื่องจากอินเทอร์เฟซและแนวคิดของระบบถูกออกแบบมาเพื่อนักพัฒนาโดยเฉพาะ

โครงการนี้จึงมุ่งหวังที่จะสร้างแพลตฟอร์มที่คล้าย GitHub แต่ปรับให้เข้าถึงง่ายและเหมาะสมกับทุกสายงาน เพื่อให้การทำงานเป็นระบบมากขึ้น โดยผสมผสานแนวคิดการบริหารจัดการโค้ด (Source Code Management) เข้ากับการจัดการงานทั่วไป ทำให้ทุกคนสามารถใช้ประโยชน์จากระบบ Workflow ที่มีประสิทธิภาพสูงได้โดยไม่ต้องมีพื้นฐานด้านการพัฒนาซอฟต์แวร์

วัตถุประสงค์ของโครงการ

1. พัฒนาเว็บแอปพลิเคชันด้วย Node.js สำหรับจัดการและกระจายงานขนาดใหญ่ให้เป็นงานย่อย (Micro-tasks)
2. สร้างระบบที่ช่วยให้การทำงานเป็นทีมมีความโปร่งใส ตรวจสอบได้ และเป็นระเบียบ คล้ายการจัดการ Source Code
3. ประยุกต์ใช้ความรู้ด้าน Data Structure ในการจัดการลำดับความสำคัญและโครงสร้างของงาน
4. ศึกษาและฝึกฝนกระบวนการพัฒนาซอฟต์แวร์แบบ Agile (Scrum) และการควบคุมคุณภาพด้วย Automated Testing

ประโยชน์ที่คาดว่าจะได้รับ

---

5. แพลตฟอร์ม GutHib จะช่วยให้บุคคลทุกสายงานสามารถบริหารจัดการงานในรูปแบบที่เป็นระบบ ตรวจสอบได้ และมีประสิทธิภาพสูง โดยไม่ต้องอาศัยความเชี่ยวชาญด้านเทคนิค
6. คณะผู้จัดทำจะได้รับความรู้และประสบการณ์จริงในการพัฒนาซอฟต์แวร์ตามหลักการ Software Engineering โดยเฉพาะกระบวนการ Agile และการทดสอบซอฟต์แวร์

ขอบเขตของโครงงาน

ในการทำโครงการครั้งนี้ ทางคณะผู้จัดทำมีเป้าหมายเพื่อสร้างแพลตฟอร์มบริหารจัดการงานสไตล์ Repository โดยขอบเขตของระบบมีดังต่อไปนี้

- ระบบผู้ใช้งาน: รองรับผู้ใช้งาน 2 กลุ่มหลัก คือ ผู้สั่งงาน (Requester) และผู้รับผิดชอบงาน (Worker)
- การจัดการงาน: สามารถสร้างโปรเจกต์หลักและซอยย่อยเป็น Issue/Task ได้
- ระบบสถานะ: งานจะมีสถานะ Open, In Progress, Review และ Closed
- การแสดงผล: รองรับการทำงานบน Web Browser (Responsive Design สำหรับ Mobile)

Requirement

Functional Requirement

Task Management (Repository Style)

- ระบุรายละเอียดงานผ่าน Markdown ได้
- รองรับการสร้างโปรเจกต์หลักและ Issue ย่อยในแต่ละโปรเจกต์

Automated Task Splitting

- แตกงานย่อยด้วยโครงสร้างข้อมูลแบบ Tree
- รองรับการจัดลำดับความสำคัญของงานแบบอัตโนมัติ

Task Claiming System

- ค้นหาและกดรับงานเพื่อสร้างกิ่งการทำงาน (Branch)
- รองรับการ Assign งานตรงไปยัง Worker คนใดคนหนึ่ง (Direct Assignment)

Proof-of-Work Submission

- ส่งผลงานพร้อมแนบหลักฐาน
- รองรับการอัปโหลดไฟล์และการเขียนรายงานผ่าน Markdown

Quality Control & Approval (Merging)

- ระบบ Accept งานและ Merge เข้าโปรเจกต์หลัก
- รองรับการ Request Changes และแสดง Diff ของงาน

Performance Analytics

- คำนวณสถิติ เช่น ค่าเฉลี่ยเวลาในการทำงาน ค่าตอบแทน และ Completion Rate

Non-Functional Requirement

- Test Coverage 100% (Unit Testing): ทุก Function ต้องมี Unit Test ครอบคลุม 100%
- Scalability (Distributed Support): ระบบต้องจัดการ Concurrency และรองรับผู้ใช้งานพร้อมกันได้หลายคน
- Auditability & Traceability: บันทึกประวัติการทำงานทุกขั้นตอน (Commit Log) เพื่อให้ตรวจสอบได้
- User Interface Responsiveness (Low Latency): หน้าเว็บต้องตอบสนองรวดเร็ว รองรับ Responsive Design บนมือถือ
- Security (Role-based Access): ระบบต้องควบคุมสิทธิ์การเข้าถึงตามบทบาทของผู้ใช้งาน

Requirement ที่เปลี่ยนไป

ไม่มีการเปลี่ยนแปลง Functional Requirement และ Non-Functional Requirement หลักจาก Phase 1 ยกเว้นการเพิ่มเติมระบบ Direct Assignment ดังนี้

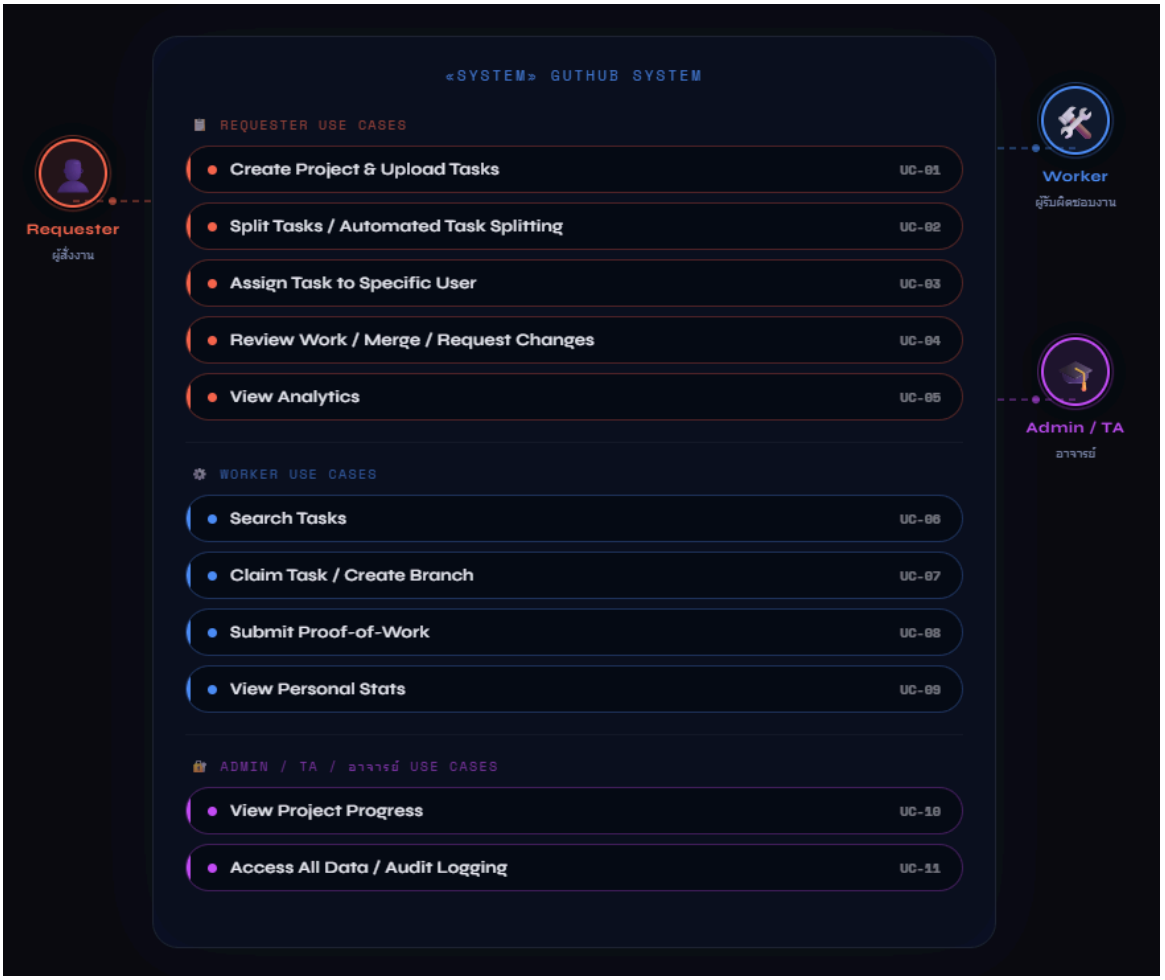
การเปลี่ยนแปลงที่เพิ่มเติม: ระบบ Direct Assignment

รายละเอียด: เพิ่มความสามารถให้ผู้สั่งงาน (Requester) สามารถ Assign งานเจาะจงไปยัง
Worker คนใดคนหนึ่งได้โดยตรง แทนที่จะรอให้ Worker กดรับงานเอง

เหตุผล: เพื่อเพิ่มความยืดหยุ่นในกรณีที่เป็นงานเฉพาะทาง หรือมีผู้รับผิดชอบที่ตกลงกันไว้แล้ว
ทำให้ลดขั้นตอนการค้นหาและกดรับงาน (Claim) ของ Worker ลง และช่วยให้การติดตาม
ความรับผิดชอบชัดเจนยิ่งขึ้น

Design Document

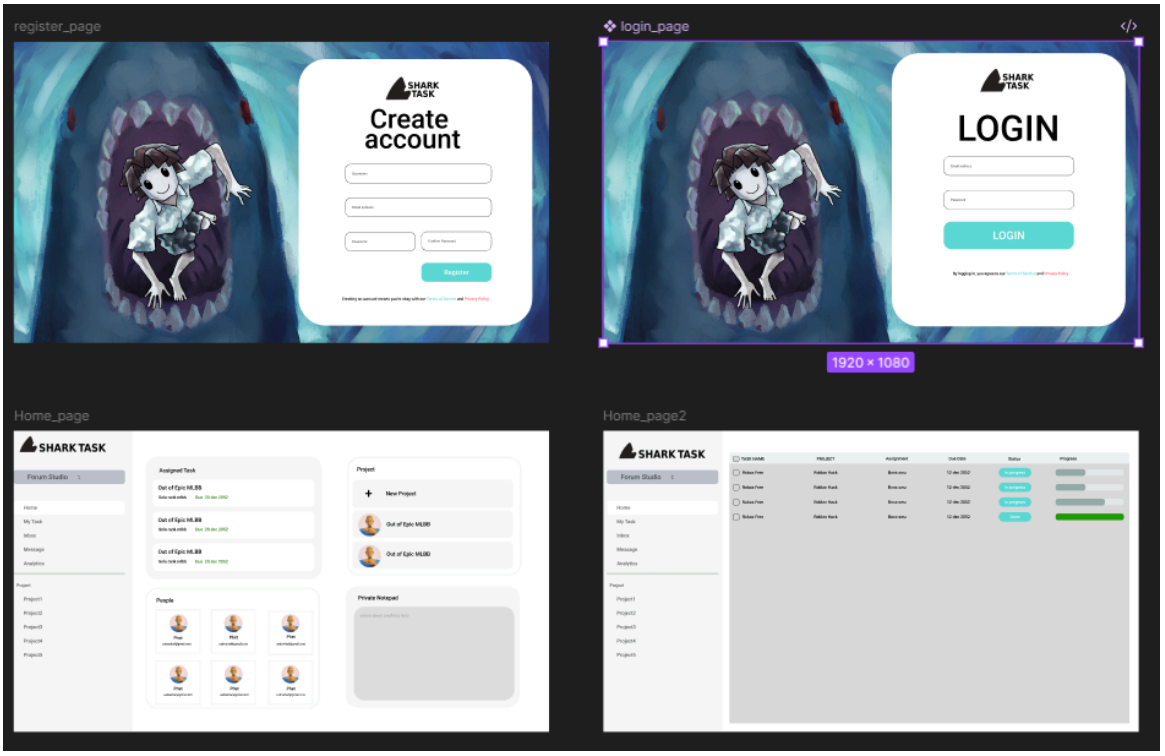
Use Case Diagram



โดย Actor แต่ละกลุ่มมีความสามารถดังนี้

- Requester (ผู้สั่งงาน): สร้างโปรเจกต์, แตกงานย่อย, มอบหมายงาน, รีวิวและ Merge งาน, ดูรายงานสถิติ
- Worker (ผู้รับผิดชอบงาน): ค้นหางาน, กดรับงาน, ส่งหลักฐาน, ดูสถิติส่วนตัว
- Admin / TA / อาจารย์: ดูความคืบหน้าโปรเจกต์ทั้งหมด, เข้าถึงข้อมูลและ Audit Log

UI DESIGN


register_page
SHARK TASK
Create account
Register
login_page
SHARK TASK
LOGIN
LOGIN
1920 × 1080
Home_page
SHARK TASK
Forum Studio
Home_page2
SHARK TASK
Forum Studio

สิ่งที่เปลี่ยนแปลง (จาก Phase 1)

---

1. เพิ่มระบบ Direct Assignment

มีการปรับปรุงให้ผู้สั่งงาน (Requester) สามารถ Assign งานเจาะจงไปยังตัว User (Worker) คนใดคนหนึ่งได้โดยตรง

เหตุผล: เพื่อเพิ่มความยืดหยุ่นในกรณีที่เป็นงานเฉพาะทาง หรือมีผู้รับผิดชอบที่ตกลงกันไว้แล้ว ทำให้ลดขั้นตอนการค้นหาและกดรับงาน (Claim) ของ Worker ลง และช่วยให้การติดตามความรับผิดชอบชัดเจนยิ่งขึ้น

กระบวนการทำงานใหม่ (Process, Methods, and Tools)

ทีมใช้กระบวนการพัฒนาแบบ Agile (Scrum Framework) โดยมีรายละเอียดดังนี้

1. ระยะเวลาและการจัดการ Sprint

ความยาว Sprint (Sprint Duration): กำหนดไว้ที่ 2 สัปดาห์ (2 Weeks) ต่อ 1 Sprint

พิธีกรรม Scrum (Scrum Events):

- มีการทำ Weekly Stand-up meeting เพื่ออัปเดตสถานะงานและปัญหา (Blocker)
- จัด Sprint Planning ก่อนเริ่ม Sprint และจัด Retrospective เมื่อจบ Sprint
- อ้างอิงวิดีโอ Retrospective: https://youtu.be/vkzCbIviAh4?si=ToOB4ghBk_POxdea

2. เครื่องมือและวิธีการ Track Status งาน

- ใช้ GitHub Projects (Kanban Board) ในการจัดการ Product Backlog และ Sprint Backlog
- จัดการแบ่งสถานะงานผ่านคอลัมน์ชัดเจน (Todo, In Progress, In Review, Done)

3. กฎการตั้งชื่อ Branch (Branch Naming Convention)

ทีมตกลงใช้รูปแบบการตั้งชื่อ Branch เพื่อให้ง่ายต่อการติดตามและสอดคล้องกับ Issue โดยใช้โครงสร้าง:

```
<ประเภทของการทำงาน>/<รหัส-Issue>-<ชื่องานสั้นๆ>

ตัวอย่าง:
  feat/GH-01-login-system          (สำหรับการสร้างฟีเจอร์ใหม่)
  fix/GH-05-button-bug             (สำหรับการแก้บั๊ก)
  docs/GH-08-update-readme         (สำหรับการเขียนหรือแก้เอกสาร)
  refactor/GH-10-restructure-api   (สำหรับการปรับปรุงโค้ดโดยไม่เปลี่ยนฟังก์ชัน)
```

4. กฎการเขียน Commit Message (Commit Message Format)

ทีมใช้มาตรฐาน Conventional Commits เพื่อให้ประวัติการทำงานอ่านง่าย และนำไปทำ Automated Release Note ได้ โดยใช้รูปแบบ:

```
<type>(<scope>): <subject>

ตัวอย่างการใช้งาน:
  feat(auth): เพิ่มระบบลงชื่อเข้าใช้ด้วย JWT
  fix(ui): แก้ไขปุ่มกดรับงานไม่แสดงผลบนมือถือ
  docs(readme): เพิ่มวิธีการรันโปรเจกต์ในเครื่อง local
  style(css): จัดรูปแบบโค้ดหน้า dashboard ใหม่
```

5. ช่องทางและวิธีการสื่อสารในทีม

- ใช้ Discord เป็นช่องทางการสื่อสารหลักในทีม
- มีการรวบรวม Requirement และเก็บข้อมูลด้วยการสัมภาษณ์แบบวิดีโอ

-VDO Restrospective

https://youtu.be/y-te4MaSvxY